S.NO	Course Code: 23CSH-309/ 23ITH-309	Full Stack -II	L	T	P	CH	C	Course Category	Course Type
1		**Course Coordinator**: Er. Tushar Sood	1	0	4	5	3	Graded(GR)	Hybrid(DE)

Pre-Requisites:	React.js, Core Java	Program Code :- CS201, CS202,CS703,CS704, IT201, IT202, BI519
Co-Requisite	Spring	
Anti-Requisite	Nill	

A. Course Description:

This course provides practical training in building modern full-stack applications using React.js and Spring Boot. Students learn front-end state management, UI design, testing, and optimization, along with back-end API development, database integration, and secure authentication using Spring Security. The course equips learners to develop scalable, secure, and industry-ready applications.

B. Course Objectives:

- Develop advanced React front-end applications with modern state management and UI libraries.
- Build secure, scalable Spring Boot back-end services with REST APIs and authentication.
- Gain practical experience in DevOps, containerization, CI/CD, and cloud deployment.
- Integrate React, Spring Boot, and cloud tools to build and deploy full-stack projects.
- Understand full-stack architecture, including JWT authentication, Redux, and microservices.

C. Course outcomes:

CO No	Statement	Performance Indicator	Student Outcome Indicator (ABET)	Level of Learning (Highest BT Level)	Target Attainment
CO1	Students will be able to understand advanced front-end applications using modern state management, UI component libraries, and performance optimization techniques.	1a,1b,2a,2b,2c	SO1, SO2	Easy (BT1)	60% (2)
CO2	Students will be able to build and secure scalable back-end services using microservices architecture, serverless functions, real-time communication, and authentication mechanisms.	1a,1c,2a,2b,4b	SO1, SO2, SO4	Medium (BT3)	60% (2)
CO3	Students will be able to integrate and manage database operations effectively within full-stack applications, ensuring efficient data handling and security.	1a,1c,2a,2c,4a	SO1, SO2, SO6	Medium (BT4)	60% (2)

CO4	Students will be able to implement testing, debugging, and state management solutions to deliver reliable and maintainable full-stack applications	2c,6a,6b,3d,3e	SO1, SO2, SO6	Hard (BT6)	60% (2)
CO5	Students will be able to deploy and scale full-stack applications using DevOps tools, containerization, CI/CD pipelines, and cloud platforms.	1c,2b,2c,5c,7b	SO2, SO3, SO5, SO7	Hard (BT5)	60% (2)

D. Syllabus /List of Experiments:

UNIT 1	- Advanced React Concepts and State Management	10Hours

Chapter- 1.1(Modern Frontend & React Fundamentals)
ES6+ syntax (Arrow functions, Spread/Rest), Functional Components, Props, State, useState, useEffect. Routing: Client-side routing with React Router, Dynamic Routes, Nested Routes, and Layout components.

Chapter- 1.2 (Advanced State Management (Context API))
Limitations of Prop-drilling, Context API (Provider/Consumer). Redux Toolkit: Store configuration, Slices, Reducers, Actions, Selectors. Fetching data from REST APIs (GET, POST, PUT, DELETE) Secure API communication (Headers, Auth tokens).

Chapter- 1.3 (Advanced UI Design)
Material UI (Grid, Typography, Theming), Ant Design (Complex Tables, Forms). Styled Components: Dynamic props, Global styles, Component composition. Handling websocket data for live UI updates.

Chapter- 1.4 (Advanced UI Design)
React Rendering Cycle, Memoization (React.memo, useMemo, useCallback). Code Splitting: Lazy loading routes (React.lazy, Suspense). Consuming microservices from frontend.

Experiment 1.1	• Create a functional component-based app using useState for logic and useEffect for lifecycle events. • Build a multi-field form using Controlled Components and a custom hook useForm for validation. • Implement React Router with dynamic parameters (/user/:id) and nested routes for a layout system.	CO1
Experiment 1.2	• Implement AuthContext to manage user login sessions and toggle UI themes globally.in react • Configure a Redux Store with productsSlice to manage global application data.in our app • Use createAsyncThunk to fetch data from an API and handle loading/success/error UI states.	CO1, CO2
Experiment 1.3	• Construct a responsive dashboard using Material UI Grid and customize the theme . • Implement a complex data table with sorting/filtering using Ant Design components. • Create a reusable component library (Buttons, Cards) styled dynamically using Styled Components.	CO3
Experiment 1.4	• Optimize a heavy list component using React.memo and useMemo to prevent re-renders. • Implement Lazy Loading for main routes to improve initial load time. • Convert the app into a PWA by adding a manifest.json and registering a Service Worker for offline access.	CO2,CO4

Unit2-	**Application Testing, Spring Boot Framework & Database Persistence**	**10Hours**

Chapter- 2.1 (Testing & Debugging)
Unit Testing with Jest, Integration Testing with React Testing Library (fireEvent, render), Snapshot Testing. Debugging**:** React DevTools, Chrome Performance Profiler, Lighthouse audits, Error Boundaries.

Chapter- 2.2 (Spring Boot Architecture)
Spring Boot Starters, Auto-configuration, Dependency Injection (@Autowired, IoC). REST APIs**:** Controller-Service-Repository architecture, DTOs, Validation (@Valid), Global Exception Handling (@ControllerAdvice).

Chapter- 2.3 (Advanced Data Persistence)
Entity Mapping (@Entity, @Table), Relationships (One-to-Many, Many-to-Many), Cascade Types. Advanced Querying: JPQL, Native Queries, Criteria API, Pagination (Pageable), Sorting, Fetch Strategies (Lazy/Eager).

Experiment 2.1	• Develop Unit tests for utility functions using Jest and Integration tests for components using React Testing Library. • Perform Snapshot testing on complex UI components to track changes over time. • Audit application performance using Google Lighthouse and debug state using React DevTools	CO2 , CO4
Experiment 2.2	• Validate React components using React Testing Library by simulating real user behavior (click, type, navigation). • Examine component render cycles and state mutations with React DevTools profiler. • Inspect API calls and loading times using browser DevTools.	CO3,CO4
Experiment 2.3	• Initialize a Spring Boot project and implement Layered Architecture (Controller, Service, Repository). • Develop REST endpoints with DTOs and implement validation annotations (@NotNull, @Size). • Create a Global Exception Handler (`@ControllerAdvice`) to return structured JSON errors	CO2,CO4

Unit3	**Application Security, Full-Stack Integration & DevOps**	**10 Hours**

Chapter-3.1 (Application Security)
Security Filter Chain, Authentication vs. Authorization. JWT: Token generation, Signing, Validation Filters. RBAC & OAuth2**:** Role-Based Access Control (`@PreAuthorize`), Google OAuth2 integration, Session management.

Chapter-3.2 (Full Stack Integration)
CORS Configuration, Axios/Fetch setup with Interceptors. **Secure Communication:** Attaching JWT to headers, Handling `401 Unauthorized`, Refresh Token flows, Error synchronization between frontend and backend.

Chapter-3.3 (DevOps & Cloud)
Docker Engine, Multi-stage Dockerfiles, Docker Compose orchestration. Pipelines: GitHub Actions (Build, Test, Deploy). Cloud: AWS EC2 deployment, Firebase/Netlify hosting, Load Balancing basics.

Experiment 3.1	• Implement a login endpoint that authenticates users and generates a signed JWT. • Create a generic Security Filter to validate JWTs on incoming requests. • Implement Role-Based Access Control (RBAC) to protect Admin routes using @PreAuthorize	CO2,CO3,CO4
Experiment 3.2	• Configure CORS in Spring Boot to allow requests from the React development server. • Create an Axios instance with interceptors to automatically attach the JWT to headers. • Implement a "Refresh Token" flow to maintain user sessions without reclogging.	CO2,CO4,CO5

Experiment 3.3	Lab Based Mini Project	CO1,CO2, CO3,CO4, CO5

E . ContentBeyondSyllabus:

Advanced Front-End Development

- **Modern React Patterns:** Implementing advanced component patterns for scalable UI development.
- **Performance Optimization:** Enhancing rendering efficiency using memoization, lazy loading, and advanced hooks.

Cloud & Microservices Development

- **Spring Boot Microservices:** Building distributed applications using Spring Boot and Spring Cloud.
- **Spring Security:** Implementing secure authentication and authorization in Spring applications.
- **REST APIs:** Designing and consuming RESTful services for full-stack integration.

DevOps & Deployment

- **Docker & Kubernetes:** Containerizing applications and managing deployments using Kubernetes.
- **Cloud Platforms:** Deploying applications on AWS, Google Cloud, and Azure for scalable production

F. Content Beyond Lab:

Experiment 1: Developing advanced React components using custom hooks and complex state logic.
Experiment 2: Implementing global state management using Context API + Reducer pattern.
Experiment 3: Building a Redux Toolkit–based feature with async thunks and API integration.
Experiment 4: Creating a responsive dashboard interface using Material UI / Ant Design.
Experiment 5: Converting a React app into a Progressive Web App (PWA) with offline support.
Experiment 6: Developing Spring Boot REST APIs with validation, DTO mapping, and exception handling.
Experiment 7: Implementing JWT authentication and role-based access control in Spring Boot.
Experiment 8: Containerizing full-stack applications using Docker and Docker Compose.
Experiment 9: Deploying a Spring Boot microservice on AWS EC2 with cloud-hosted database integration.
Experiment 10: Setting up a CI/CD pipeline using GitHub Actions for automated build & deployment.
Experiment 11: Implement Distributed Caching with Redis in Spring Boot
Experiment 12: Deploy a Spring Boot microservice using Kubernetes rolling updates
Experiment 13: Apply rate limiting to protect backend services.
Experiment 14: Add logging, metrics, and tracing to a full-stack system
Experiment 15: Extract one module of your backend into a microservice while keeping React UI functional
Experiment 16: Build a secure file-upload system using Spring Boot + S3 + React.
Experiment 17: Integrate WebSockets between React UI and Spring Boot backend.
Experiment 18: Create an automated CI/CD pipeline for a React + Spring Boot application
Experiment 19: Use Redis caching to improve performance of expensive backend operations
Experiment 20: Implement a multi-tenant React + Spring Boot application where different users access isolated datasets

G. Textbooks:

a. (David Choi),Full-Stack Web Development with React and Node, Packt Publishing, International Edition, 2024
b. (Marten Deinum),Learning Full-Stack JavaScript Development, O'Reilly Media, Standard Edition, 2023
c. (Magnus Larsson),Spring Boot and Spring Cloud Microservices, Packt Publishing, 2nd Edition, 2024

H. Referencebooks:

d. (JamesGosling), KenArnoldandDavidHolmes,Java Programming Language, 5th Edition, Pearson Education.

e. (GaryCornell),"CoreJava", VolumeI, 3rd Edition, Pearson Education.

I. Assessment Pattern—Internal and External

	Theory		Practical	
Components	**Internal Assessment (CAE)**	**Semester End Examination (SEE)**	**Continuous Internal Assessment (CAE)**	**Semester End Examination (SEE)**
Marks	40	60	60	40
Total Marks	100		100	

J. Internal and External Evaluation Components(Theory)

Sno.	**Direct Evaluation Instruments**	**Weightage of actual conduct**	**Frequency of Task**	**Final Weightage in Internal Assessment**	**BT Levels**	**CO Mapping**	**Mapping with SIs (ABET)**	**Mapping with PIs**	**Remarks (Graded/Non-Graded)**
1	Assignment	10 marks for each assignment	One per unit	6	Hard	CO4,CO5	SO1,SO6	1a,1b,1c	Graded
2	EST	60 marks for one EST	1 per semester	30	Medium	CO1,CO2,CO3,CO4,CO5	SO1,SO6	7a,7b	Graded
3	MST 1	20 marks for MST1	1 per semester	5	Medium	CO1,CO2,CO3	SO1,SO6	1a,1d	Graded
4	MST 2	20 marks for MST2	1 per semester	5	Medium	CO3,CO4,CO5	SO1,SO6	1c,1d	Graded
5	Quiz/Test	4 marks for quiz	1 per sem	4	Easy	CO1,CO2	SO1	1c,1d	Graded
6	Surprise test	12 marks for each test	One per unit	4	Medium	CO2,CO3	SO1	1c,1d	Graded
7	Homework	NA	One per lecture topic (of 2 questions)	NA	NA	NA	NA	NA	Non-Graded
8	Case study	NA	NA	NA	NA	NA	NA	NA	Non-Graded

S No.	Direct Evaluation Instruments	Weightage of actual conduct	Frequency of Task	Final Weightage in Internal Assessment	BT Levels	CO Mapping	Mapping with SIs (ABET)	Mapping with PIs	Remarks
1	Worksheet	30 marks for each worksheet	8-10 experiments	20	Medium	1,2,3,4,5	1a, 1b, 1c, 6a, 6b	SO1, SO6	Graded
2	Lab MST	10 marks for one MST	1 per semester	4	Medium	1,2,3	1a, 1b, 1c	SO1, SO6	Graded
3	Practical Evaluation	40 marks for one external practical	1 per semester	20	Medium,Hard	1,2,3,4,5	1a, 1b, 1c, 6a, 6b	SO1, SO6	Graded

K. Internal and External Evaluation Components(Practical)

Program Name	Course Name	Assessment Name	Exam Description	WMM	Max Marks
CS201 :: Bachelor of Engineering - Computer Science & Engineering	23CSH-309 :: Full Stack Development - II	Hybrid Course All	Assignment/PBL	6	10
CS201 :: Bachelor of Engineering - Computer Science & Engineering	23CSH-309 :: Full Stack Development - II	Hybrid Course All	Attendance Marks	2	2
CS201 :: Bachelor of Engineering - Computer Science & Engineering	23CSH-309 :: Full Stack Development - II	Hybrid Course All	End Term Hybrid Theory	30	60
CS201 :: Bachelor of Engineering - Computer Science & Engineering	23CSH-309 :: Full Stack Development - II	Hybrid Course All	MST-1 Hybrid	5	20
CS201 :: Bachelor of Engineering - Computer Science & Engineering	23CSH-309 :: Full Stack Development - II	Hybrid Course All	MST-2 Hybrid	5	20
CS201 :: Bachelor of Engineering - Computer Science & Engineering	23CSH-309 :: Full Stack Development - II	Hybrid Course All	Practical Evaluations	20	40
CS201 :: Bachelor of Engineering - Computer Science & Engineering	23CSH-309 :: Full Stack Development - II	Hybrid Course All	Practical MST	4	10
CS201 :: Bachelor of Engineering - Computer Science & Engineering	23CSH-309 :: Full Stack Development - II	Hybrid Course All	Practical Worksheet/Projects 1	2	30
CS201 :: Bachelor of Engineering - Computer Science & Engineering	23CSH-309 :: Full Stack Development - II	Hybrid Course All	Practical Worksheet/Projects 10	2	30

CS201 :: Bachelor of Engineering - Computer Science & Engineering	23CSH-309 :: Full Stack Development - II	Hybrid Course All	Practical Worksheet/Projects 2	2	30
CS201 :: Bachelor of Engineering - Computer Science & Engineering	23CSH-309 :: Full Stack Development - II	Hybrid Course All	Practical Worksheet/Projects 3	2	30
CS201 :: Bachelor of Engineering - Computer Science & Engineering	23CSH-309 :: Full Stack Development - II	Hybrid Course All	Practical Worksheet/Projects 4	2	30
CS201 :: Bachelor of Engineering - Computer Science & Engineering	23CSH-309 :: Full Stack Development - II	Hybrid Course All	Practical Worksheet/Projects 5	2	30
CS201 :: Bachelor of Engineering - Computer Science & Engineering	23CSH-309 :: Full Stack Development - II	Hybrid Course All	Practical Worksheet/Projects 6	2	30
CS201 :: Bachelor of Engineering - Computer Science & Engineering	23CSH-309 :: Full Stack Development - II	Hybrid Course All	Practical Worksheet/Projects 7	2	30
CS201 :: Bachelor of Engineering - Computer Science & Engineering	23CSH-309 :: Full Stack Development - II	Hybrid Course All	Practical Worksheet/Projects 8	2	30
CS201 :: Bachelor of Engineering - Computer Science & Engineering	23CSH-309 :: Full Stack Development - II	Hybrid Course All	Practical Worksheet/Projects 9	2	30
CS201 :: Bachelor of Engineering - Computer Science & Engineering	23CSH-309 :: Full Stack Development - II	Hybrid Course All	Quiz	4	4
CS201 :: Bachelor of Engineering - Computer Science & Engineering	23CSH-309 :: Full Stack Development - II	Hybrid Course All	Surprise Test	4	12

L. CO-PO Mapping

COvsPO/PSO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10
CO1	3	3	3	2	2	3	3	2	1	3
CO2	3	3	2	2	2	3	2	2	2	3
CO3	2	2	2	2	2	2	2	2	1	2
CO4	2	2	2	2	2	2	2	1	2	2
CO5	3	3	2	3	2	3	2	3	2	3
AVG	**2.6**	**2.6**	**2.2**	**2.2**	**2.0**	**2.6**	**2.2**	**2.0**	**1.6**	**2.6**

COvsPO/PSO	PO11	PO12	PO13	PO14	PO15	PO16	PO17	PSO1	PSO2
CO1	3	1	2	2	1	1	1	2	2
CO2	3	2	2	2	1	1	1	2	2
CO3	2	1	2	2	1	2	1	2	2
CO4	2	1	2	2	2	1	1	2	2
CO5	3	2	2	3	2	2	3	3	3
AVG	**2.6**	**1.4**	**2.0**	**2.2**	**1.4**	**1.4**	**1.4**	**2.2**	**2.2**

ProgramOutcomes(POs)

PO 1: Disciplinary knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialisation for the solution of complex engineering problems.

PO 2: Complex problem solving

PO 3: Critical Thinking

PO 4:Creativity-Create, perform, or think in different and diverse ways about the given scenarios

PO 5: Communication skills: Communicate effectively on complex engineering activities with the engineering community and with the society at large, such as being able to comprehend and Develop effective reports and design documentation, make effective presentations, and give and receive clear instructions

PO 6: Analytical reasoning/thinking: Identify, formulate, research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO 7: Research related skills: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for public health and safety, and cultural, societal, and environmental considerations.

PO 8: Coordinating/Collaborating with others: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO 9: Leadership qualities: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal, and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO 10: Learning how to learn skills: Recognise the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PO11: Digital and technological skills: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO 12: Multicultural Competence and Inclusive Spirit

PO 13: Value inculcation: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO 14: Autonomy, responsibility and accountability: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO 15: Environmental awareness and action: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and the need for sustainable development.

PO 16: Community Engagement & Services

PO 17: Empathy 17.1 Identify with or understand the perspective, experiences, or points of view of another individual or group, and to identify and understand other people's emotions

Program Specific Outcomes (PSOs)

PSO 1. Exhibit attitude for continuous learning and deliver efficient solutions for emerging challenges in the computation domain.

PSO 2. Apply standard software engineering principles to develop viable solutions for Information